Practical No. 1

Mapped CO: CO1 Unit I & II Bloom Level: L3 – Apply

1.0) AIM

Write a program to generate a hash and simulate a simple digital signature process using SHA-256.

2.0) SCOPE

Program to generate a hash using SHA-256 and to simulate a digital signature process (signing with a private key and verifying with a public key), demonstrating data integrity and authenticity as used in blockchain systems.

3.0) FACILITIES

- **Computer System:** Any desktop or laptop computer with basic configuration.
- **Software Package:** Python 3 (IDLE), with the built-in hashlib library and the ecdsa library (install using: pip install ecdsa).

4.0) THEORY

What is SHA-256?

SHA-256 (Secure Hash Algorithm 256-bit) is a cryptographic hash function that produces a fixed-size 256-bit (32-byte) hash. It is part of the SHA-2 family and is widely used for secure data integrity and authentication.

Characteristics of SHA-256:

1. **Fixed Size Output:** Regardless of the input size, SHA-256 produces a 256-bit hash.
2. **Deterministic:** The same input will always produce the same hash.
3. **Fast Computation:** Efficiently computes the hash of the input data.
4. **Preimage Resistance:** It is computationally infeasible to reverse-engineer the original input from its hash.
5. **Collision Resistance:** It is highly unlikely to find two different inputs that produce the same hash.
6. **Avalanche Effect:** A small change in input significantly changes the output hash.

Practical Use Cases:

1. **Data Integrity:** Verifying that data has not been altered.
2. **Password Storage:** Storing hashed passwords to enhance security.
3. **Digital Signatures:** Ensuring the authenticity and integrity of digital documents.
4. **Blockchain:** Ensuring the integrity and security of transactions.

What is a Digital Signature?

A digital signature proves who created a message and confirms that the message has not been changed. It uses a pair of keys: a private key, which is kept secret and used to sign, and a public key, which is shared and used to verify. The sender signs the message with the private key; anyone can then verify it with the matching public key. If the message is altered after signing, verification fails. This is exactly the mechanism Bitcoin and Ethereum use to authorise transactions, and it relies on the same secp256k1/elliptic-curve cryptography used in this program.

Steps to Implement SHA-256 Hashing in Python:

1. **Import the hashlib Library:** The hashlib library provides a common interface to secure hash algorithms, including SHA-256.
2. **Create a SHA-256 Hash Object:** Use hashlib.sha256() to create a new hash object.
3. **Update the Hash Object:** Feed the input data into the object using update(); string input must be encoded to bytes.
4. **Retrieve the Hash:** Use hexdigest() to get the hexadecimal hash (or digest() for raw bytes).

Steps to Simulate a Digital Signature:

1. **Generate a key pair:** Create a private key and its matching public key using the ecdsa library.
2. **Sign the message:** Use the private key to produce a signature for the message.
3. **Verify the signature:** Use the public key to check the signature; it passes for the original message and fails if the message is tampered with.

5.0) PROGRAM

Part A — SHA-256 Hashing and the Avalanche Effect

```
import hashlib

def hash_string_sha256(input_string):
    sha256_hash = hashlib.sha256()
    sha256_hash.update(input_string.encode('utf-8'))
    return sha256_hash.hexdigest()

# Hash a sample string
input_string = "Hello, world!"
print("Original string :", input_string)
print("SHA-256 hash    :", hash_string_sha256(input_string))

# Avalanche effect: change just one character and re-hash
changed_string = "Hello, world."   # '!' changed to '.'
print("\nChanged string  :", changed_string)
print("SHA-256 hash    :", hash_string_sha256(changed_string))
```

Part B — Digital Signature (Sign with Private Key, Verify with Public Key)

```
from ecdsa import SigningKey, SECP256k1

# 1. Generate a key pair (same curve used by Bitcoin & Ethereum)
private_key = SigningKey.generate(curve=SECP256k1)
public_key = private_key.get_verifying_key()

message = "This is a confidential message."

# 2. SIGN the message using the PRIVATE key
signature = private_key.sign(message.encode('utf-8'))
print("Message   :", message)
print("Signature :", signature.hex()[:40], "...")

# 3. VERIFY the signature using the PUBLIC key
try:
    public_key.verify(signature, message.encode('utf-8'))
    print("Verification (original message) : VALID")
except Exception:
    print("Verification (original message) : INVALID")
```

```
# 4. Tamper with the message and verify again(Optional)
tampered = "This is a tampered message."
try:
    public_key.verify(signature, tampered.encode('utf-8'))
    print("Verification (tampered message) : VALID")
except Exception:
    print("Verification (tampered message) : INVALID")
```

6.0) EXPECTED OUTPUT

Part A prints the 64-character SHA-256 hash of the sample string, and shows that changing a single character ('!' to '.') produces a completely different hash — demonstrating the avalanche effect.

```
Original string : Hello, world!
SHA-256 hash    : 315f5bdb76d078c43b8ac0064e4a0164612b1fce77c869345bfc94c75894edd3

Changed string  : Hello, world.
SHA-256 hash    : f8c3bf62a9aa3e6fc1619c250e48abe7519373d3edf41be62eb5dc45199af2ef
```

Part B prints the signature, then confirms the signature is VALID for the original message and INVALID for the tampered message — demonstrating that the digital signature guarantees both authenticity and integrity.

```
Message   : This is a confidential message.
Signature : 3045022100b8f2e1c4a9... ...
Verification (original message) : VALID
Verification (tampered message) : INVALID
```

Note: *The signature value differs on every run because a new random key pair is generated each time the program executes.*

7.0) CONCLUSION

Students will understand how SHA-256 hashing ensures data integrity (through the deterministic, avalanche-sensitive hash) and how a digital signature, using a private key to sign and a public key to verify, ensures both authenticity and integrity — the same cryptographic foundation on which blockchain transactions are built.